

Calculo de Peso dos Arquivos para Backup

Este documento explica como o sistema decide se um arquivo tem prioridade baixa, media ou alta no processo de backup, com base no codigo atual do projeto.

1. Ideia geral

O sistema nao olha o conteudo real do arquivo para decidir o peso. Em vez disso, ele usa metadados e sinais observaveis, como nome, extensao, contexto da pasta, quantidade de modificacoes, quantidade de acessos e duplicidade por hash.

A prioridade nasce de uma pontuacao. O arquivo comeca com uma base fixa e vai ganhando ou perdendo pontos conforme esses sinais. No fim, a soma e convertida para uma faixa de prioridade:

- 0 a 39 : prioridade **baixa**
- 40 a 69 : prioridade **media**
- 70 a 100 : prioridade **alta**

2. De onde os sinais saem

Antes de calcular o peso, o scanner coleta varias informacoes do arquivo. Entre as mais importantes estao:

- `modified_count` : quantas vezes o arquivo aparenta ter mudado entre varreduras

- `accessed_count` : quantas vezes o arquivo aparenta ter sido acessado entre varreduras
- `days_since_modified` : ha quantos dias o arquivo nao e alterado
- `extension` : extensao do arquivo, como `pdf` , `xlsx` , `db` , `py`
- `directory_context` : contexto da pasta onde ele esta
- `important_keyword` : se o nome contem palavras como `contrato` , `projeto` , `financeiro` ou `documento`
- `is_duplicate` : se outro arquivo do scan possui o mesmo hash

Os contadores de modificacao e acesso nao sao fornecidos pelo usuario. O proprio scanner os constroi ao comparar os metadados atuais do arquivo com os metadados vistos em scans anteriores.

3. Pontuacao base

O calculo sempre comeca com `score = 10` . Essa base evita que o arquivo comece em zero absoluto, mas so se torna relevante quando outros sinais entram na conta.

4. Sinais que aumentam o peso

Sinal	Peso	Como funciona
Modificacoes frequentes ou recentes	+18	Entra quando <code>modified_count >= 2</code> ou <code>days_since_modified <= 2</code>
Extensao com alto valor para backup	+18	Exemplos: <code>docx</code> , <code>pdf</code> , <code>xlsx</code> , <code>db</code> , <code>sql</code> , <code>json</code> , <code>py</code>
Extensao critica para sistema ou programa	+20	Exemplos: <code>env</code> , <code>db</code> , <code>json</code> , <code>yaml</code> , <code>py</code> , <code>ps1</code> , <code>pem</code>

Sinal	Peso	Como funciona
Acessos frequentes	+14	Entra quando <code>accessed_count >= 2</code>
Nome importante	+24	Quando o nome sugere algo valioso, como <code>cliente</code> , <code>contrato</code> , <code>financeiro</code> , <code>senha</code> , <code>projeto</code>
Contexto importante de pasta	+18	Quando o caminho sugere relevancia, como <code>Documents</code> , <code>Projetos</code> , <code>Clientes</code> , <code>Financeiro</code> , <code>src</code>
Palavra-chave importante detectada pelo scanner	+10	Usa a lista local de palavras-chave ja configurada no scanner

5. Sinais que reduzem o peso

Sinal	Peso	Como funciona
Arquivo duplicado	-6	Quando o hash do arquivo ja apareceu antes no mesmo scan
Extensao descartavel	-18	Exemplos: <code>tmp</code> , <code>cache</code> , <code>log</code> , <code>old</code>
Contexto temporario ou tecnico	-35	Exemplos: <code>__pycache__</code> , <code>.git</code> , <code>node_modules</code> , <code>venv</code> , <code>temp</code>

6. Faixas finais de prioridade

Depois de somar tudo, a pontuacao final e limitada entre `0` e `100` . Isso impede resultados negativos ou pontuacoes acima de cem.

- **Alta:** quando o score final e `70` ou mais
- **Media:** quando o score final fica entre `40` e `69`

- **Baixa:** quando o score final fica abaixo de **40**

7. Exemplos praticos

Exemplo 1: `contrato_clientes_2026.xlsx` em uma pasta como `Documents > Empresa > Contratos`, modificado recentemente.

- Base: **10**
- Modificacao recente: **+18**
- Extensao valiosa: **+18**
- Nome importante: **+24**
- Contexto importante: **+18**

Total aproximado: **88**. Resultado: prioridade **alta**.

Exemplo 2: `debug.log` em uma pasta como `venv > logs`.

- Base: **10**
- Extensao descartavel: **-18**
- Contexto temporario: **-35**

Total aproximado: vai para perto de **0**. Resultado: prioridade **baixa**.

8. Onde a Gemini entra

A Gemini nao apaga a regra local. O sistema sempre calcula a pontuacao local primeiro. Depois, se a LLM estiver ativa, ele envia os metadados para a Gemini e recebe uma opiniao adicional em formato estruturado.

Se a Gemini responder com boa confianca, a prioridade final pode ser refinada. Se ela falhar ou responder com baixa confianca em desacordo com a regra local, o sistema tende a preservar a regra local como protecao.

Isso faz a arquitetura ser hibrida: primeiro regra objetiva, depois interpretacao semantica. Assim o sistema fica mais estavel do que uma decisao tomada so pela LLM.

9. Como a prioridade impacta o backup

A prioridade final nao serve apenas para exibir uma etiqueta. Ela controla quando o arquivo volta a ser elegivel para backup automatico pela politica por prioridade:

- **Baixa:** backup semanal
- **Media:** backup a cada 2 dias
- **Alta:** backup no primeiro inicio do dia e depois a cada 4 horas, desde que o arquivo tenha mudado

Para arquivos de alta prioridade, o sistema ainda verifica se o hash mudou antes de gerar uma nova copia no intervalo de 4 horas. Isso evita repeticao inutil.

10. Resumo conceitual

A melhor maneira de entender o peso e pensar que o sistema nao pergunta apenas “este arquivo e importante?”. Em vez disso, ele pergunta:

- ele muda muito?
- ele e acessado com frequencia?
- a extensao sugere valor tecnico ou documental?
- o nome sugere conteudo importante?
- a pasta reforca que esse arquivo importa?
- ha sinais de que ele e temporario, descartavel ou duplicado?

O peso final e a resposta acumulada dessas perguntas.

Documento gerado a partir da implementacao atual do projeto `sistema-backup-inteligente` .